

Java Project Report

Chalumuri Shiva Rohit (cs24btech11018)

April 2025

Introduction

This is the report for the java arbitrary precision project. Basically we have 2 java files AInteger.java and AFloat.java which contain the arithmetic operation methods on these classes. We have done this project by taking string representations of these numbers and manipulating them.

Design

Basic

We have a arbitraryarithmetic package consisting of these 2 classes AFloat and AInteger. Also a MyInfArith class which has main method to test these. The reason we did this all using strings was to handle arbitrarily large numbers without overflow. Also because of implicit type conversion involving string concatenation.

Project Structure

```
/my_project
build/
    MyInfArith.class
    arbitraryarithmetic/
        AFloat.class
        AInteger.class
src/
```

```
MyInfArith.java
MyInfArith.class
arbitraryarithmetic/
    AFloat.java
    AFloat.class
    AInteger.java
    AInteger.class
arbitraryarithmetic/
    aarithmetic.jar
run_project.py
build.xml
README.md
report.pdf
```

AInteger class

This is the AInteger class which represents integers as strings. It is being used for arbitrary arithmetic precision on integers.

Instance variables

1. String value
2. Boolean isnegative

Constructors

1. Default constructor AInteger() to initialise to 0
2. Constructor to instantiate an object AInteger(String s)
3. Copy constructor AInteger(AInteger obj)

Static and getter methods

1. Method to turn a string into our object AInteger parse(String s)
2. Getter for value String getvalue()
3. Getter for isnegative boolean getisnegative()

Various helpers

These are to be used in actual arithmetic methods. All of these are static as they are not used on objects but of the class itself

1. `String removefrontzeros(String s)` : Method to remove leading zeros from a string
2. `String[] padzeros(String a, String b)` : Method adds zeros to the shorter string of the two to make them of equal length
3. `boolean isSmaller(String a, String b)` : Method returns true if a is numerically smaller than b
4. `String addstr(String a, String b)` : Method to add strings simply based on magnitude
5. `String subtractstr(String a, String b)` : Method to subtract strings based on digit by digit math
6. `String digitwisemultiply(String num, char digitchar)` : Multiplies a string with a single digit
7. `String multiplystr(String a, String b)` : Method to multiply two strings based on digit wise multiply used above
8. `String dividestr(String dividend, String divisor)` : Method to divide two integers based on repeated subtraction

Operation methods

These are the actual methods to be called on the objects created

1. `AInteger add(AInteger other)` : Method for addition
2. `AInteger subtract(AInteger other)` : Method for subtraction

3. AInteger multiply(AInteger other) : Method for multiplication
4. AInteger divide(AInteger other) : Method for division

UML representation

This is a visual representation of the design approach in the class

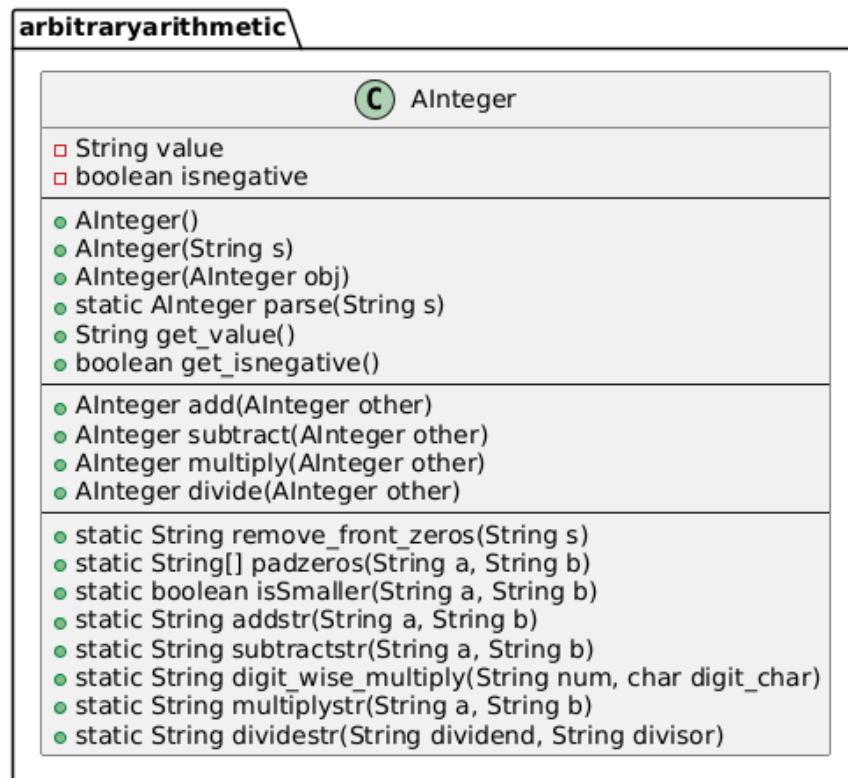


Figure 1: UML of AInteger

AFloat class

This is the AFloat class which represents floats as strings. It is being used for arbitrary arithmetic precision on floats.

Instance variables

1. String value
2. Boolean isnegative

Constructors

1. Default constructor AFloat() to initialise to 0
2. Constructor to instantiate an object AFloat(String s)
3. Copy constructor AFloat(AFloat obj)

Static and getter methods

1. Method to turn a string into our object AFloat parse(String s)
2. Getter for value String getvalue()
3. Getter for isnegative boolean getisnegative()

Various helpers

These are to be used in actual arithmetic methods. All of these are static as they are not used on objects but of the class itself

1. String removefrontzeros(String s) : Method to remove leading zeros from a string
2. String removeendzeros(String s) : Method to remove trailing zeros
3. String[] padfloat(String a, String b) : Method adds zeros to the shorter string of the two to make them of equal length

4. String insertdecimalpoint(String s, int decimalplaces) : Method to insert a decimal point based on number of places where required

5. boolean issmaller(String a, String b) ; Method returns true if a is smaller

6. String truncate(String result) : Method to truncate upto required amount 30 in our case.

Operation methods

These are the actual methods to be called on the objects created

1. AFloat add(AFloat other) : Method for addition

2. AFloat subtract(AFloat other) : Method for subtraction

3. AFloat multiply(AFloat other) : Method for multiplication

4. AFloat divide(AFloat other) : Method for division

UML representation

This is a visual representation of the design approach in the class

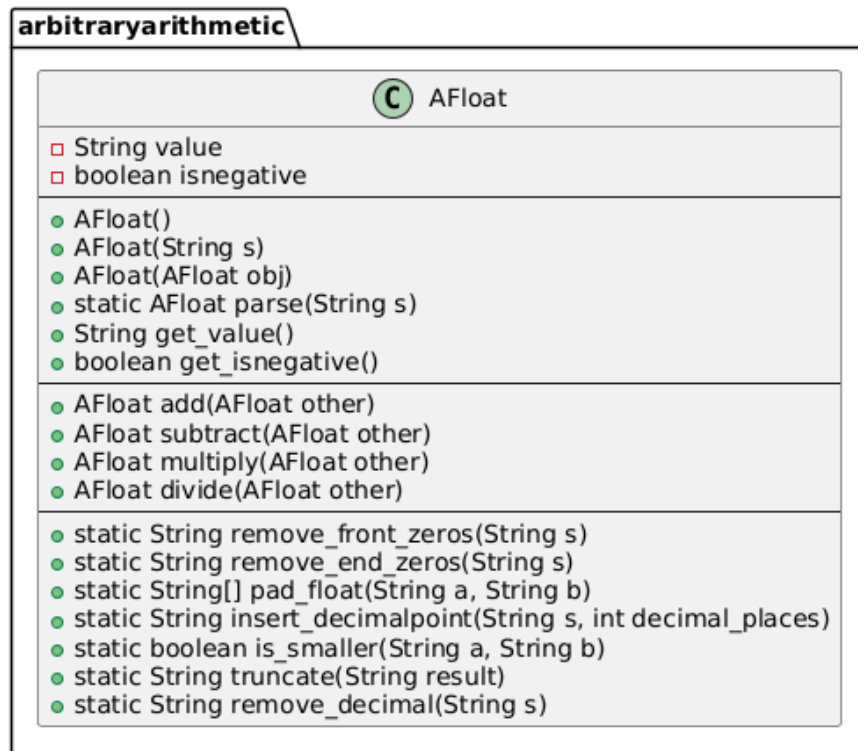


Figure 2: UML of AFloat

MyInfArith class

This is the class which has the main method in it . It is used for taking 4 command line arguments and prints the output of the result to the terminal (screen).

Methods present

1. void main(String[] args) : The entry point of our java program. Has the command line arguments as its input
2. dointegeroperation(String operation, String num1, String num2) : Cre-

ates the two AInteger objects does the method on them and prints output.

3.dofloatoperation(String operation, String num1, String num2) : Creates the two AFloat objects does the method on them and prints output.

run project.py

This is the python script which compiles the .java files into .class files and runs the commands as in like python3 scriptname type operation num1 num2

The compiled .class files also lie in the src directory of root.

build.xml

This is the makefile for this project. It can also be used to run the test cases in form of ant run command.

It compiles the .java files into .class files into the build directory and the ant jar command gives the arbitraryarithmetic/aarithmetic.jar which can be linked to any executable.

report.pdf

It is this report itself being written in latex.

README.MD

Explains what the project does its purpose, and any context behind it.Used to describe and document the project

LIMITATIONS

1. This project wont work well for float division in case of having very small input like a single digit and divisor being arbitrarily large it simply gives 0.0

as output.

For eg :
java MyInfArith float div 1 1000000000000000027738383994949404040
0.0

Verification approach

This can be done in 3 ways to verify the program

1) using the main class itself : java MyInfArith float mul 2.5 2.0
5.0

2) using the python script : python3 runproject.py int add 3 4
7

3) using ant run command : ant run -Darg1=float -Darg2=add -Darg3=100.5
-Darg4=3.25
103.75

Key learnings

1. I understood how OOP is useful while just importing methods i have used in AInteger directly into code of AFloat.
2. Also how to encapsulate field variables and methods using setter and getter methods .
3. How to do division by using repeated subtraction manually.

GIT SNAPSHOT

```
May 2, 14:58
rohitshiva@rohitshiva-VivoBook-ASUSLaptop-K3502ZA: ~/my_project

commit 347296b4a07bac788be65f9ec2c273f79049a14f
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 11:03:36 2025 +0530

    Final AInteger class

commit d230720a66b9d9de568f6a5ef024a90eddc3831
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 10:39:39 2025 +0530
Date: Fri May 2 10:39:39 2025 +0530

    Modified the main method to catch exceptions

commit 6656182029f1ea510472a27439b00202cebe7417 (tag: state1)
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:31:03 2025 +0530

    Changed the truncate function

commit d939c4391e1b526b63d90ef9f093324dd147f817
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:13:47 2025 +0530

    The makefile

commit d9d685b3ba71ec7b2c6a0e4c8c4d9c5ff8ab1e3
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:12:27 2025 +0530

    The python script

commit bf9b4aee535feb5b6a29a96e5a346cb28ef8a1fa
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:10:58 2025 +0530

    The main class is added

commit a1419f7e0f29f0977f3b953d00e2017e9ffb655
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:08:23 2025 +0530

    AFloat is added

commit 522d7d6b6ad71dbca4d4a1543754ec6281ebfbc3
Author: rohit <cs24btech11018@lith.ac.in>
```

Figure 3: Snapshot 1

```
Initial Commit

commit 6656182029f1ea510472a27439b00202cebe7417 (tag: state1)
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:31:03 2025 +0530

    Modified the main method to catch exceptions

commit d939c4391e1b526b63d90ef9f093324dd147f817
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:13:47 2025 +0530

    Changed the truncate function

commit d9d685b3ba71ec7b2c6a0e4c8c4d9c5ff8ab1e3
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:12:27 2025 +0530

    The makefile

commit bf9b4aee535feb5b6a29a96e5a346cb28ef8a1fa
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:10:58 2025 +0530

    The python script

commit a1419f7e0f29f0977f3b953d00e2017e9ffb655
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 03:08:23 2025 +0530

    The main class is added

commit 522d7d6b6ad71dbca4d4a1543754ec6281ebfbc3
Author: rohit <cs24btech11018@lith.ac.in>
Date: Fri May 2 02:31:05 2025 +0530

    AFloat is added

commit 62aeb27719f10e979930ae7ea8cc5933cdec64f3
Author: rohitshiva23 <cs24btech11018@lith.ac.in>
Date: Fri May 2 02:12:01 2025 +0530

    Added the AInteger class

Initial commit
```

Figure 4: Snapshot 2

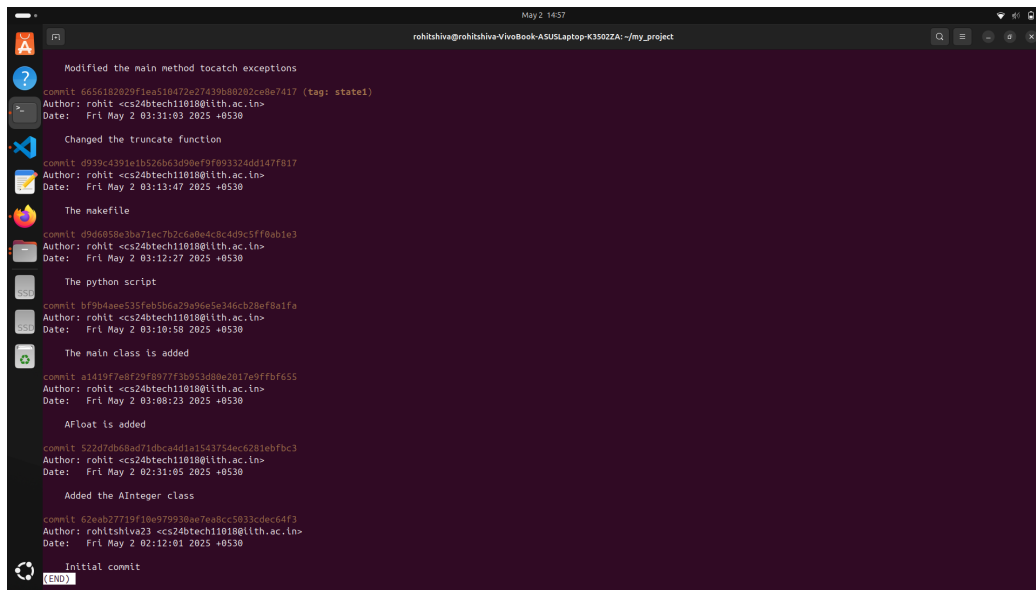


Figure 5: Snapshot 3